4 Liste

Strutture dati lineari

Le strutture lineari di dati si sviluppano in una sola dimensione e possono essere considerate come una **sequenza di oggetti**.
La loro caratteristica fondamentale è che in esse è presente una **relazione d'ordine** tra gli oggetti, che permette di individuare chi viene prima.
Questa relazione d'ordine non implica necessariamente un ordinamento dei valori (es. alfabetico o numerico), ma unicamente l'ordine posizionale.
Per distinguere le diverse strutture lineari, è rilevante considerare due fattori.

- **i modi di accedere (modalità si accesso)**: come si individua la posizione nella sequenza in cui operare.
- **I modi di agire (Operatori)**: i tipi di azione che si possono compiere sulla posizione individuata.

Metodi di accesso:

- **Accesso diretto:** Consente di accedere al singolo componente mediante il **nome** e il meccanismo dell'**indice**
  - **Esempio il vettore (o array):** il vettore è il tipico esempio di struttura dati astratta che corrisponde alla struttura dati fisica in C (struttura dati concreta).
- **Accesso a scansione (sequenziale)**: consente l'accesso all'elemento generico solo dopo aver **scandito** sequenzialmente gli elementi che lo precedono. Si ha un punto di ingresso e dei meccanismi per muoversi sequenzialmente (avanti o indietro).
  - **Esempio tipico:** la Lista.
- **Accesso a Estremi:** strutture che permettono di accedere solo al primo elemento e/o all'ultimo.
  - **Esempio tipico:** la **pila** (che usa l'algoritmo LIFO - last in First out) e la **coda** (che usa l'algoritmo FIFO - First in First out).

Relativamente ai modi di agire nelle posizioni, le operazioni operazioni basilari in una sequenza sono:

- **Ispezione (Lettura)**: Lettura del valore di un componente.
- **Aggiornamento (Cambio di valore)**: Aggiornamento del valore di un componente.
- **Inserzione (Scrittura)**: Inserimento di un nuovo componente.
- **Rimozione (cancellazione)**: Rimozione di un componente.
  Nelle strutture dati le operazioni da fare sono generalmente basilari.

LISTE

Definizione:
Una lista è definita una sequenza **finita, anche vuota (λ)**, di elementi dello stesso tipo (omogenei).
La lista viene indicata con

$$L =< a_1, a_2, \ldots, a_n >$$

- A differenza del concetto di insieme, nella lista uno stesso elemento può comparire più volte in posizioni diverse.
- La Lista è una struttura dati **dinamica** perchè non ha una dimensione prefissata e può crescere e decrescere nel tempo, rendendo fondamentali le operazioni di inserimento e cancellazione che alterano la dimensione (a differenza del vettore).
- **Posizione e valore:** a ciascun elemento a_i di una lista viene associato:
  - una posizione $pos(i)$
  - un valore $a(i)$
- **Accesso:** si può accedere direttamente solo al primo elemento della sequenza. Per accedere al generico elemento, occorre **scandire sequenzialmente** gli elementi che lo precedono.
- **Operazioni**: la lista è a **dimensione variabile**, per questo si possono eseguire **l'inserimento** e la **cancellazione**, queste due operazione vanno a modificare la dimensione, concetto in ammissibile per i vettori.

Lunghezza e sottoliste

La lunghezza di una lista è il **numero dei suoi elementi**.
La lunghezza conta le posizioni, non i simboli distinti.
Una lista è detta vuota quando il numero di elementi è zero. La lista vuota si indica con `<>` ed essa è sottolista di qualsiasi lista.

Sottolista: Data una lista

$$L =< a_1, a_2, \ldots, a_n >$$

una sottolista è una sequenza

$$< a_i, \ldots, a_j >$$

ottenuta partendo da una posizione i e prendendo tutti gli elementi fino alla posizione j, senza salti, con $1 \leq i \leq j \leq n$.

Specifica della lista (Sintattica e semantica)

TIPI ASTRATTI

Tipo	Definizione
Lista	Insieme delle sequenze $L =< a_1, a_2, \ldots, a_n >$, $n \geq 0$, di elementi di tipo tipoelem, dove l'elemento i-esimo ha valore $a(i)$ e posizione $\text{pos}(i)$.

Tipo	Definizione
Posizione	(Tipo astratto per indicare la posizione).
Boolean	Insieme dei valori di verità (True/False, 1/0).
Tipoelem	Il tipo degli elementi contenuti nella lista.

OPERATORI (specifica semantica)

Tabella riassuntiva degli operatori principali con relativa sintassi, pre-condizione e post-condizione.

Operatore	Sintassi	Pre-condizione (PRE)	Post-condizione (POST)
crealista	crealista(): lista	Nessuna	L′ = <> (λ - sequenza vuota)
listavuota	listavuota(L): boolean	Nessuna	b = 1 (true) se L = <>; b = 0 (false) altrimenti
leggilista	leggilista(p, L): tipoelem	p = pos(i), con 1 ≤ i ≤ n	a = a(i)
scrivilista	scrivilista(a, p, L): lista	p = pos(i), con 1 ≤ i ≤ n	L′ = $<a_1,\ldots,a_{i-1},a,a_{i+1},\ldots,a_n>$
primolista	primolista(L): posizione	Nessuna	p = pos(1)
finelista	finelista(p, L): boolean	p = pos(i), con 1 ≤ i ≤ n + 1	b = 1 (true) se p = pos(n + 1); b = 0 (false) altrimenti
succlista	succlista(p, L): posizione	p = pos(i), con 1 ≤ i ≤ n	q = pos(i + 1)
predlista	predlista(p, L): posizione	p = pos(i), con 2 ≤ i ≤ n + 1	q = pos(i − 1)
inslista	inslista(a, p, L): lista	p = pos(i), con 1 ≤ i ≤ n + 1	L′ = $<a_1,\ldots,a_{i-1},a,a_i,\ldots,a_n>$ se 1 ≤ i ≤ n. L′ = $<a_1,\ldots,a_n,a>$ se i = n + 1. (implica L′ = <a> se i = 1 e L = <>)
canclista	canclista(p, L): lista	p = pos(i), con 1 ≤ i ≤ n	L′ = $<a_1,\ldots,a_{i-1},a_{i+1},\ldots,a_n>$

Implicazioni della specifica semantica
Dalla specifica semantica emerge che per accedere a un elemento occorre conoscerne la **posizione**

- **Accesso diretto limitato:** possibile solo per il **primo elemento** della lista.
- **Operatore di posizione:** l'unico operatore che reatituisce direttamente la posizione del primo elemento è *primolista*.
- **Scansione Obbligatoria**: Per tutti gli elementi, la posizione si ottiene conoscendo a priori la posizione dell'elmento precedente (o seguente)e applicando l'operatore *succlista* (o *predlista*). Dunque per accedere a un generico elemento occorre **scandire tutta la lista a partire dal primo elemento**.
- **Ridondanza:** l'operatore *listavuota* è ridondante perchè può essere sostituito dalla composizione di operatori *finelista (primolita)*.

Realizzazione Sequenziale con Vettore

Una lista può essere implementata utilizzando un **vettore (array monodimensionale)**.

- **Struttura**: Poiché la lista è dinamica, la realizzazione sequenziale usa variabili per per poter gestire la lista, detti:
  - **primo:** variabile per l'indice della componente del vettore in cui è memorizzato il primo elemento della lista.
  - **lunghezza:** variabile per indicare il **numero effettivo di elementi** di cui è composta la lista rappresentata.
- **Vantaggio:** consente di realizzare in modo molto semplice alcune delle operazioni definite per la lista (es. *primolista, succlista* sono a complessità $O(1)$).
- **Problema:** il vero problema riguarda le operazioni di *inlista* (inserzione) e *canclista* (rimozione). L'inserzione o cancellazione di un elemento causa lo **spostamento** di tutti gli elementi successivi all'interno del vettore per mantenere la contiguità fisica, rendendo l'operazione costosa ($O(n)$).
  La banale infezione in una terza posizione di un nuovo elemento come nell'esempio,

causa lo spostamento verso il basso del quarto e quindi elemento.

Rappresentazione collegata

L'idea fondamentale della rappresentazione collegata è quella di memorizzare gli elementi della lista associando ad ognuno di essi una particolare informazione (detta **riferimento o puntatore**).
Questo riferimento permette di individuare dove è memorizzato l'elemento successivo della sequenza logica.
Per visualizzare tale rappresentazione si usa la notazione grafica in cui:

- Gli elementi sono rappresentati mediante **nodi**.
- I riferimenti sono rappresentati mediante **archi** che collegano i nodi.

Come si può notare:

- Si usa un **riferimento iniziale** al primo elemento della lista.
- Si usa un simbolo speciale, ∅ (insieme vuoto) o NULL, come riferimento associato all'ultimo nodo. Nel caso la lista sia vuota, tale simbolo compare direttamente nel riferimento iniziale.

Realizzazione con cursori

La realizzazione con cursori utilizza un vettore (array monodimensionale) per l'implementazione della lista, ma sfrutta i riferimenti per superare il problema dell'aggiornamento (inserimento o cancellazione di un elemento) che affligge la realizzazione sequenziale pura.
I riferimenti si realizzano mediante i **cursori**, cioè variabili intere o enumerative il cui valore è interpretato come indice di una cella del vettore.
Si definisce un unico vettore, detto **vettore spazio** che:

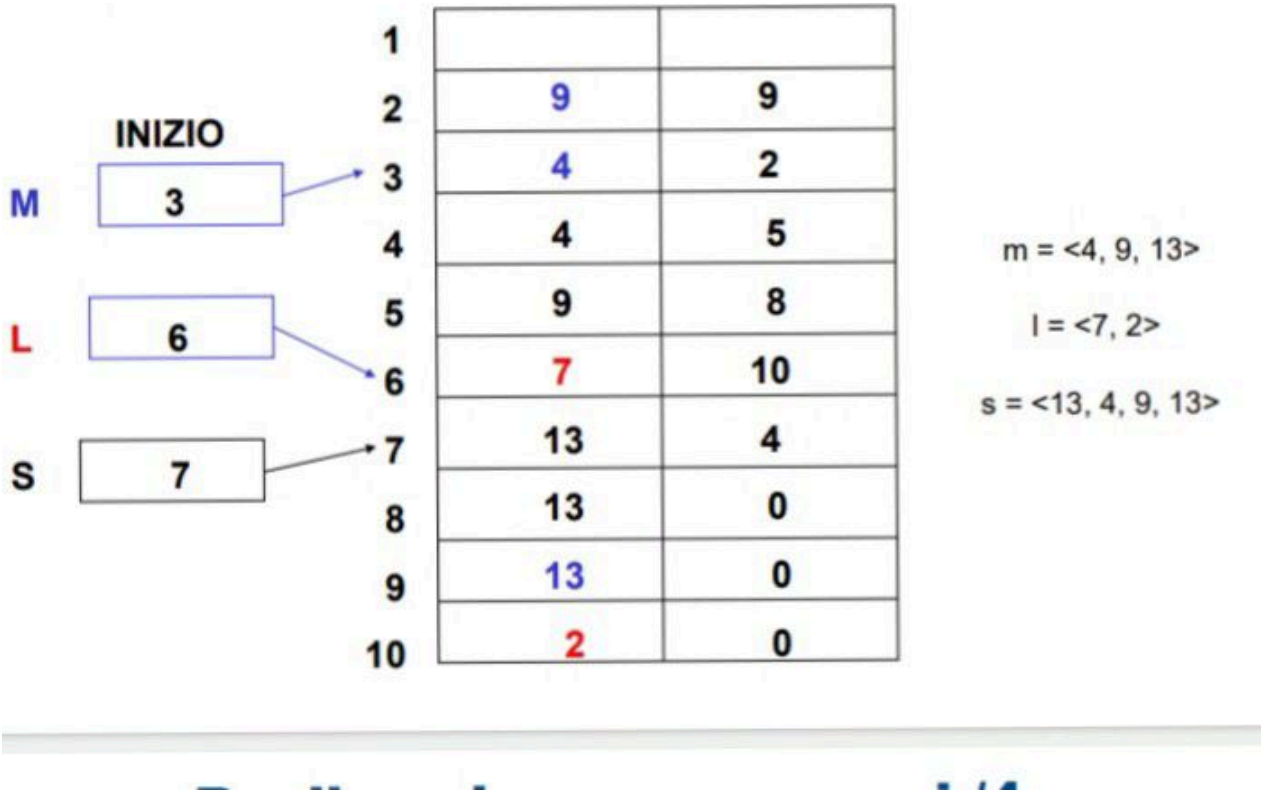
- Può contenere più liste distinte, ognuna individuata da un proprio cursore iniziale (es. INIZIO L, INIZIO M, INIZIO S).
- Contiene tutte le celle libere, organizzate in una lista aggiuntiva della *listalibera*.

Esempio:
Supponiamo di avere tre diverse liste, l m,s, dove:
$l =< 7, 2 > m =< 4, 9, 13 > s =< 13, 4, 9, 13 >$
Per rappresentare queste tre liste possiamo usare un unico vettore.
Struttura della cella: La componente (cella) del vettore spazio ha due campi:

- Elemento: dove è memorizzato il contenuto del nodo.
- Successivo: contiene il cursore (indice) ovvero il riferimento al prossimo nodo logico della lista.

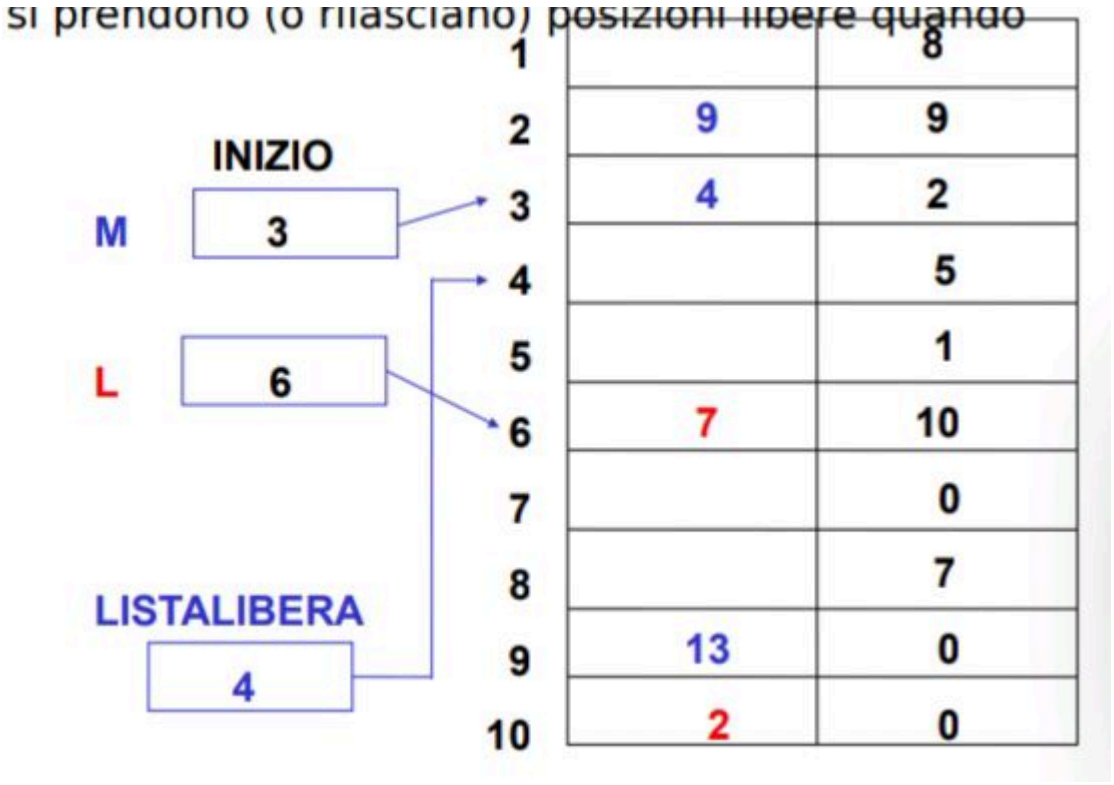
  Posizione Logica: La sequenza degli elementi che formano la lista è ricostruibile iniziando dalle componente dell'array corrispondente al valore di inizio, nel cui campo **elemento** è memorizzato il valore del loro primo elemento.
  La posizione astratta $pos(i)$ è uguale al valore del cursore alla cella del vettore spazio che contiene l'elemento i-esimo di l che è uguale al valore del cursore alla cella del vettore spazio che contiene l'elemento $(i-1)esimo$, se $2 \leq i \leq n+1$,uguale a 0 se $i = 1$.
  Analogamente:
- pos(n+1) è il cursore alla cella successiva all'elemento n-esimo (se n≥1), o 0 altrimenti.

- La lista vuota si indica con L=∅ (insieme vuoto).



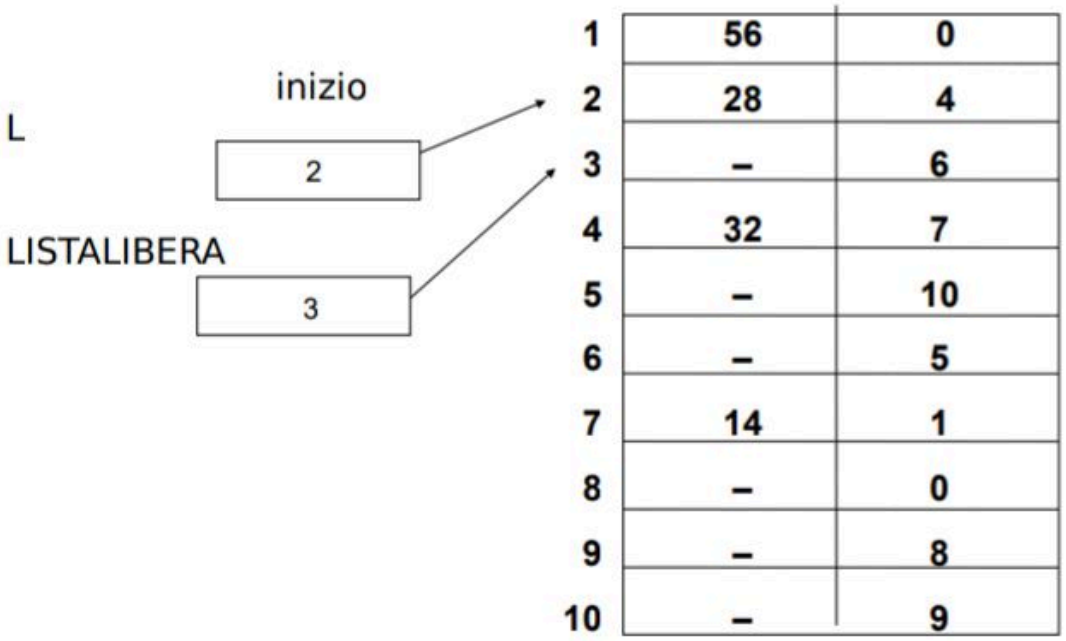
Aggiornamento e listalibera: Per inslista e canclista, il problema di individuare una posizione libera è risolto usando la **listalibera**.

- listalibera è memorizzata nello stesso array e raccoglie in modo collegato le componenti libere.
- Essa funge da **serbatoio** da cui prelevare (per l'inserimento) e in cui riversare (per la cancellazione) le componenti inutilizzate.

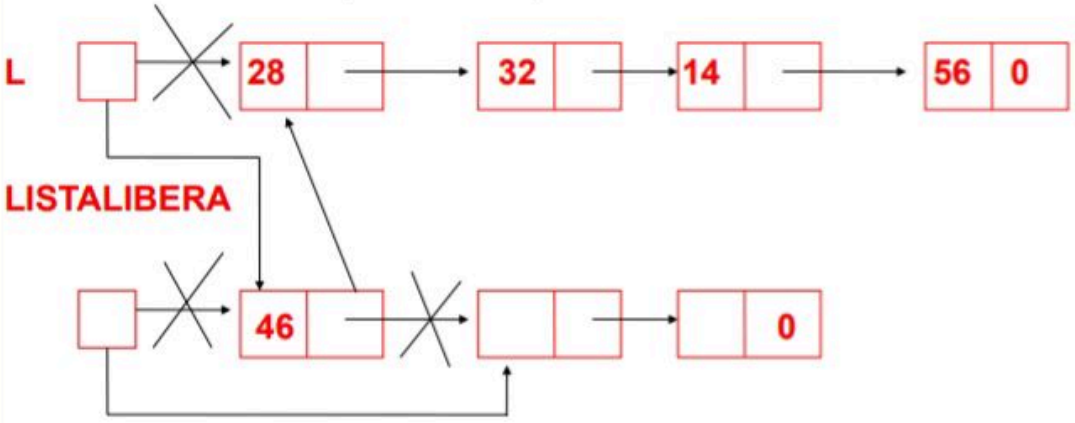


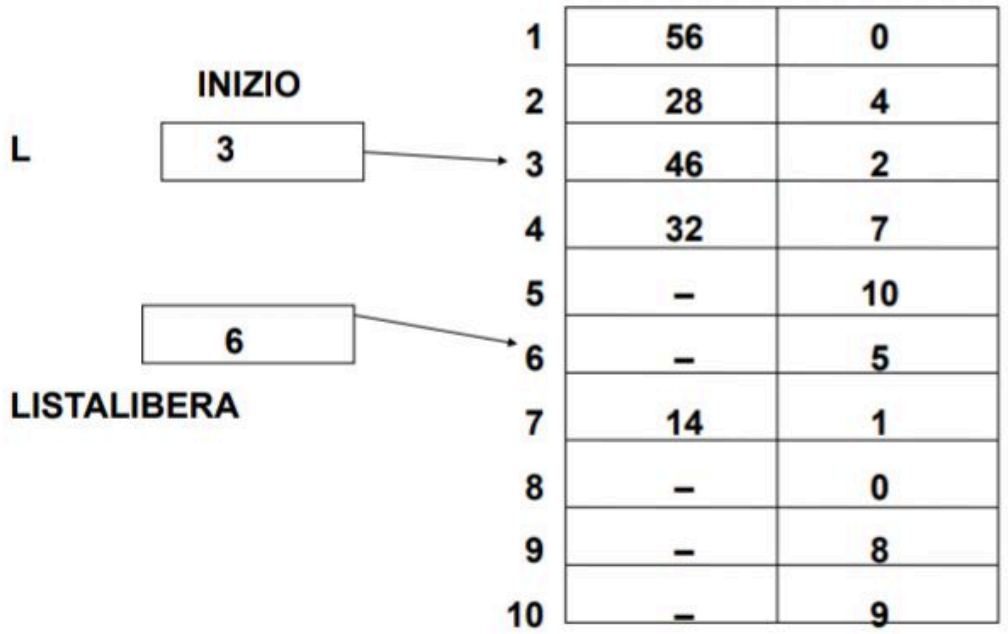
- La listalibera rappresenta un serbatoio da cui prelevare componenti libere dell'array e in cui riversare le componenti dell'array che non sono piu' utilizzate per la lista

L = [28, 32, 14, 56]



- Se si vuole inserire un nuovo elemento (= 46) in testa alla lista si fa uso della prima componente della lista libera





È immediato verificare che grazie all'uso della *listalibera*, non richiedono lo spostamento di altri elementi della lista le operazioni di inserimento e cancellazione. Tuttavia restano i problemi legati all'uso dell'array, cioè l'**esigenza di definire una dimensione**.

Vantaggio/Svantaggio della rappresentazione sequenziale: L'inserimento e l'eliminazione **non richiedono lo spostamento** di altri elementi, superando un grosso svantaggio della realizzazione sequenziale pura. La complicazione data dalla listalibera è compensata dal fatto che gli aggiornamenti sono veloci ($O(1)$). Tuttavia, **rimangono gli svantaggi connessi all'uso dell'array**:
1. La dimensione dell'array è un **limite massimo** alla crescita della lista.
2. La memoria utilizzata **non dipende dalla lunghezza effettiva** della lista.
3. C'è un'ulteriore **occupazione di memoria** per memorizzare i riferimenti (campo "successivo").

Realizzazione con Puntatori

La realizzazione con puntatori è considerata la più **efficace** realizzazione della rappresentazione collegata.
Si basa sull'uso congiunto del **tipo puntatore e del record (o struct)**.

- Una variabile di tipo puntatore *p* memorizza l'indirizzo di una locazione di memoria. Le operazioni fondamentali su un puntatore sono:
  1. **Accesso**: Accesso alla locazione in cui è memorizzato l'indirizzo di *p*.
  2. **New**: richiesta di una nuova locazione di memoria e memorizzazione del suo suo indirizzo in *p*.

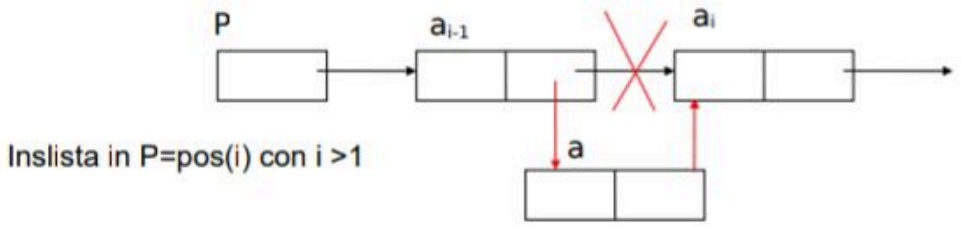
3. **Delate**: Rilascio della locazione di memoria il cui indirizzo è memorizzato in *p*.
   -Una possibile realizzazione è una **lista monodirezionale semplificata;** in questa realizzazione, si ha una struttura di n elementi o "**celle**". L'i-esima cella contiene:
   - L'i-esimo elemento della lista.
   - L'**indirizzo** della cella che contiene l'elemento successivo (il puntatore).

- **Gestione degli indirizzi:**
  - La prima cella è indirizzata da una variabile l di tipo **puntatore**
  - L'ultima cella punta a un valore convenzionale NULL.
  - Gli indirizzi sono noti alla macchina **ma non al programmatore**.
  - La posizione astratta $pos(i)$ è uguale al valore puntato alla cella che contiene l'i-esimo elemento, con $1 \leq i \leq n$.

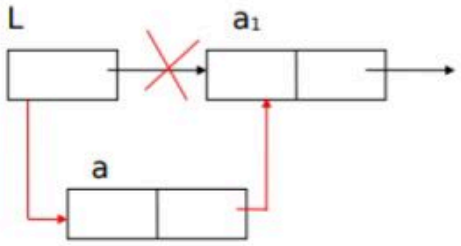
Argomento: relazione d'ordine e vettore

La relazione s'ordine che definisce una struttura lineare (come vettore o la lista) è puramente **posizionale** (tipologica): a_i precede a_{i+1}.
Questo è sempre vero. Al contrario, un **ordinamento** si riferisce ai calori degli elementi, come ad esempio $a_i \leq a_{i+1}$. Un vettore non ordinato conserva la relazione d'ordine posizionale (l'elemento in posizione 5 viene dopo quello di posizione 4) anche le proprietà.
La lista standard, in quanto sequenziale, conserva solo la relazione d'ordine posizionale, garantita dai riferimenti logici (cursore o puntatore).

L'aggiornamento dei puntatori



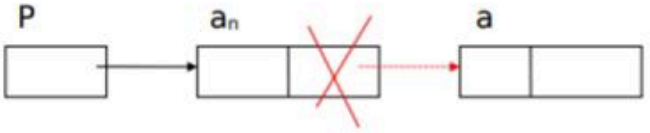
Inslista in P=pos(i) con i >1

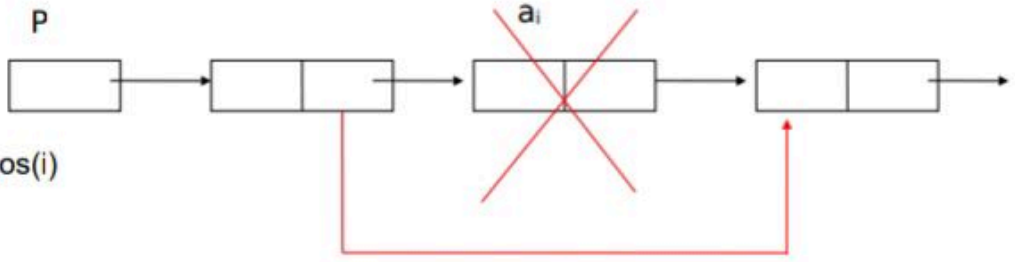


Inslista in P=pos(1)

L'aggiornamento dei puntatori /2

Inslista in P=pos(n+1)





Canclista P=pos(i)

Svantaggio della realizzazione sequenziale (vettore)

In un array, gli elementi devono essere memorizzati in locazioni di memoria contigue. Se si vuole inserire un elemento in posizione i (con $1 \leq i \leq n$), è necessario **spostare tutti gli elementi da a_i fino a a_{a+1} a a_n di una posizione indietro** per colmare il vuoto.
In entrambi i casi la complessità richiede O(n) operazioni di copia (dove n è la lunghezza della lista/vettore), rendendola inefficiente in tempo per liste lunghe.

Vantaggio con realizzazione con i puntatori

A differenza della realizzazione con cursori, che utilizza un array di dimensione fissa (il limite spazio), la realizzazione con puntatori si affida a meccanismo si **allocazione dinamica della memoria** (*new*).
Quando è necessario inserire un nuovo nodo la funzione *new* richiede al sistema operativo una nuova porzione di memoria *ovunque* essa sia disponibile e restituisce l'indirizzo (il puntatore).
La gestione della memoria libera (l'equivalente della listalibera) è demandata direttamente al sistema (o al *garbage collector*), superando il limite prefissato di un array e permettendo alla lista di crescere fino a quando la memoria del sistema lo consente.

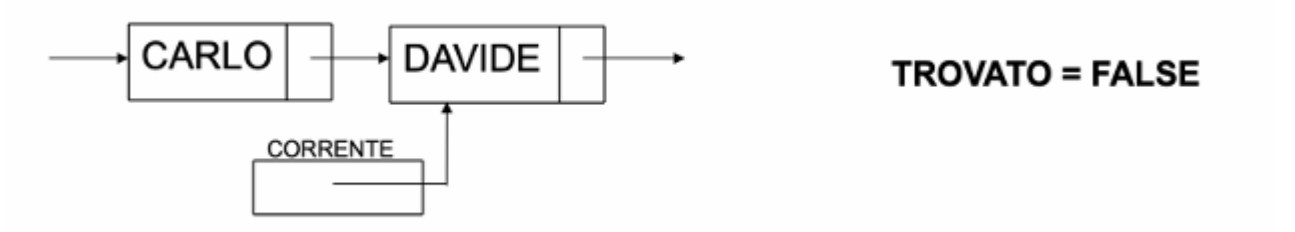
Ricerca in una Lista Lineare Ordinata

L'algoritmo di ricerca non si basa sull'indice fisico (come un array), ma sull'ordine logico, scorrendo la lista tramite l'operatore `succlissta`.

L'idea centrale è:
"mentre l'elemento che cerco è più grande dell'elemento corrente della lista, passa all'elemento successivo".
Per fare ciò, si usano due puntatori:

1. `corrente`: Punta all'elemento che stiamo analizzando ora.
2. `precedente`: Punta all'elemento analizzato al passo prima.
   È fondamentale aggiornare `precedente = corrente` *prima* di avanzare `corrente` con `corrente = succlista(corrente, I)`. Questo perché, specialmente per inserimenti e cancellazioni, abbiamo bisogno di un riferimento all'elemento *prima* della posizione trovata, per poter modificare i collegamenti.

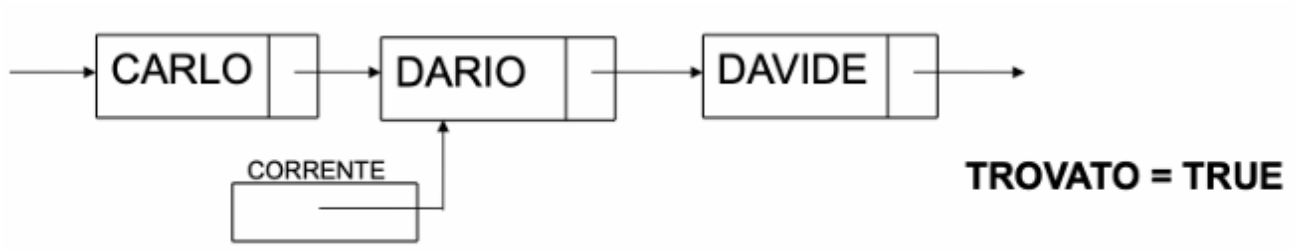
Nell'ambito della ricerca in una lista lineare ordinata, un approccio più robusto rispetto alla semplice restituzione di un valore booleano (`true` / `false`) consiste nel restituire il **puntatore (o posizione)** `corrente` al quale la scansione si è arrestata.
Questo metodo è più versatile perché il puntatore restituito assume un significato operativo preciso in *entrambi* gli scenari, sia in caso di successo che di fallimento della ricerca, come illustrato dall'esempio della ricerca di "*dario*".



L'algoritmo confronta "dario" con "CARLO" ($dario > carlo$) e avanza. Successivamente, confronta "dario" con "DAVIDE" ($dario < davide$).

- **Arresto:** La ricerca si interrompe perché ha trovato un elemento ("DAVIDE") che, nell'ordine alfabetico, è *superiore* a quello cercato.
- **Significato del puntatore:** Il puntatore `corrente` restituito punta a "DAVIDE". Questo valore non è inutile; al contrario, indica **l'esatta posizione logica prima della quale l'elemento "dario" dovrebbe essere inserito** per preservare l'ordinamento della lista.

Analizziamo questo caso:



L'algoritmo confronta "dario" con "CARLO" ($dario > carlo$) e avanza. Successivamente, confronta "dario" con "DARIO" ($dario == dario$).

- **Arresto:** La ricerca si interrompe perché ha trovato una *corrispondenza esatta*.
- **Significato del puntatore:** Il puntatore `corrente` restituito punta **direttamente all'elemento "DARIO"**. Questa è la posizione necessaria per eseguire operazioni successive sull'elemento, come la sua cancellazione (`canclista`) o la lettura di dati associati (`leggilista`).

Fusione di liste ordinate

L'algoritmo di fusione (o *merge*) ha lo scopo di **combinare due liste già ordinate** (che chiameremo `Lista1` e `Lista2`) in una **terza lista** (`Lista3`), la quale deve contenere tutti gli elementi delle prime due e deve risultare anch'essa ordinata.
Questo algoritmo è fondamentale ed è un blocco di costruzione per metodi di ordinamento più complessi, come il *Natural Merge Sort* (visto nelle slide successive).

```
creaLista(Lista3)
p1=primolista(Lista1)
p2=primolista(Lista2)
p3=primolista(Lista3)
while (not finelista(p1,Lista1) and not finelista (p2,Lista2)) do
```

```
    elem1=leggilista(p1,Lista1)
    elem2=leggilista(p2,Lista2)
    if elem1 < elem2 then
        inslista(elem1,p3,Lista3)
        p1=succlista(p1,Lista1)
    else
        inslista(elem2,p3,Lista3)
        p2=succlista(p2,Lista2)
    p3=succlista(p3,Lista3)

    while not finelista (p1,Lista1) do
        inslista(leggilista(p1,Lista1),p3,Lista3)
        p1=succlista(p1,Lista1)
        p3=succlista(p3,Lista3)
    while not finelista(p2,Lista2) do
        inslista(leggilista(p2,Lista2),p3,Lista3)
        p2=succlista(p2,Lista2)
        p3=succlista(p3,Lista3)
```

La complessità temporale dell'algoritmo di fusione (merge) è $O(n+m)$.
L'algoritmo è **lineare** rispetto al numero totale di elementi. Questo perché, per costruire la `Lista3` ordinata, l'algoritmo deve esaminare ogni elemento di `Lista1` e ogni elemento di `Lista2` **esattamente una volta**.

Logica di Funzionamento

L'algoritmo è molto efficiente perché esegue la fusione in un'unica passata, senza bisogno di riordinare la lista finale. La logica si basa su un confronto sequenziale.

1. **Inizializzazione:**
   - Si crea una `Lista3` vuota.
   - Si impostano tre puntatori (posizioni) per scorrere le liste:
     - `p1` punta al primo elemento di `Lista1`.
     - `p2` punta al primo elemento di `Lista2`.
     - `p3` punta alla prima posizione (vuota) di `Lista3`.
2. **Ciclo Principale di Confronto:**
   - Si entra in un ciclo `while` che continua **fintanto che entrambe le liste hanno ancora elementi** (`not finelista(p1, Lista1) AND not finelista(p2, Lista2)`).
   - All'interno del ciclo, si leggono i due elementi correnti: `elem1` (da `p1`) e `elem2` (da `p2`).
   - **Si confrontano i due elementi:**
     - **Se `elem1 < elem2`**: L'elemento più piccolo è `elem1`. Viene inserito in `Lista3` e si fa avanzare *solo* `p1` alla posizione successiva di `Lista1`.

    - **Altrimenti (`elem1 >= elem2`)**: L'elemento più piccolo (o uguale) è `elem2`. Viene inserito in `Lista3` e si fa avanzare *solo* `p2` alla posizione successiva di `Lista2`.
  - Indipendentemente da quale elemento sia stato inserito, `p3` **viene sempre fatto avanzare** per puntare alla prossima posizione libera in `Lista3`.
3. **Fase di "Pulizia" (Copia dei Rimanenti):**
  - Il ciclo principale termina quando una delle due liste (o entrambe) è finita.
  - A questo punto, è possibile che una delle due liste contenga ancora elementi. Questi elementi sono, per definizione, già ordinati e sono tutti più grandi degli elementi già inseriti in `Lista3`.
  - **Primo ciclo di pulizia:** Un `while` controlla se `Lista1` ha ancora elementi (`not finelista(p1, Lista1)`). Se sì, copia tutti gli elementi rimanenti da `Lista1` a `Lista3`.
  - **Secondo ciclo di pulizia:** Un `while` fa lo stesso per `Lista2` (`not finelista(p2, Lista2)`). Copia tutti gli elementi rimanenti da `Lista2` a `Lista3`.

Alla fine di questo processo, `Lista3` conterrà la fusione perfettamente ordinata di `Lista1` e `Lista2`.

Ordinamento di una Lista (Natural Merge Sort)

Questo algoritmo non guarda i singoli elementi, ma sfrutta le "catene" (chiamate anche *run*).
Una **catena** è una sottosequenza della lista che è **già ordinata al suo interno**.
Qualsiasi lista, anche se disordinata, può essere vista come una sequenza di catene.
Una catena parte dall'inizio della lista o quando l'ordine si "rompe" (es. $a_{i-1} > a_i$) e termina alla fine della lista o quando l'ordine si "rompe" di nuovo (es. $a_k > a_{k+1}$).

Esempio:
$l = 82\ 16\ 14\ 15\ 84\ 25\ 77\ 13\ 75\ 4$, dove l è ottenuta dalla concatenazione di sei catene di varia lunghezza.
Questa lista è composta da **6 catene**:

- **Catena 1:** <82> (finisce perché 82>16)
- **Catena 2:** <16> (finisce perché 16>14)
- **Catena 3:** <14, 15, 84> (finisce perché 84>25)
- **Catena 4:** <25, 77> (finisce perché 77>13)
- **Catena 5:** <13, 75> (finisce perché 75>4)
- **Catena 6:** <4> (finisce perché la lista termina)

Logica del Natural Merge Sort

L'idea di questo algoritmo è:

1. La lista è una sequenza di k catene.

2. Se usiamo l'algoritmo di **Fusione** per fondere le catene tra loro, otterremo una nuova lista con un numero minore di catene (circa $k/2$), che saranno lunghe il doppio.
3. Ripetendo questo processo di fusione più volte, il numero di catene diminuirà (k, k/2, k/4, ...) finché non si arriverà ad avere **una sola catena**.
4. Una lista composta da una sola catena è, per definizione, una lista ordinata.

Partiamo da:
l = 82, 16, 14, 15, 84, 25, 77, 13, 75, 4

1° Passaggio – Identificazione delle catene
Troviamo:
(82) | (16) | (14,15,84) | (25,77) | (13,75) | (4)

2° Passaggio – Fusione di catene consecutive
Si applica la fusione (come nell'algoritmo di "Fusione di liste ordinate"):
l = 16, 25, 77, 82, 4, 14, 15, 84, 13, 75

Le catene risultano più lunghe, ma meno numerose.

3° Passaggi successivi
Ad ogni passo:

- le catene si **distribuiscono** su due liste ausiliarie (A e B);
- si **fondono** alternativamente per ricreare la lista principale con meno catene;
- il numero di catene **si dimezza ogni volta**.

Nel caso peggiore, in cui ogni elemento sia una catena da 1 solo valore (cioè lista completamente disordinata), serviranno $log_2(n)$ fusioni.

4° Passaggio – Risultato finale
Alla fine otteniamo una sola catena:
l = 4, 13, 14, 15, 16, 25, 75, 77, 82, 84
Ogni iterazione del ciclo `repeat-until` si compone di due fasi principali: Distribuzione e Fusione.

1. Fase di Distribuzione (`distribuisci`)

- `crealista(A)` e `crealista(B)` : Vengono inizializzate (o svuotate) le due liste ausiliarie `A` e `B` .
- `distribuisci(L, A, B)` : Questa procedura esegue una scansione della lista `L` per identificare le "catene naturali" (sequenze massimali di elementi già ordinati). Queste catene vengono quindi distribuite alternativamente nelle liste `A` e `B` .
  - *Esempio:* Se `L = [3, 5, | 2, 4, 8, | 1, 9]` (le '|' indicano la fine di una catena), la procedura assegnerà `[3, 5]` ad `A` , `[2, 4, 8]` a `B` , e `[1, 9]` nuovamente ad `A` .
  - Si ottiene: `A = [3, 5, | 1, 9]` e `B = [2, 4, 8]` .

2. Fase di Fusione (`merge`)

- `numero_catene = 0`: Viene inizializzato un contatore. Questa variabile (passata per riferimento alla funzione `merge`) terrà traccia del numero di *nuove* catene create durante la fusione.
- `crealista(L)` : La lista originale `L` viene svuotata per poter accogliere i dati fusi e riordinati.
- `merge(A, B, L, numero_catene)` : Questa è la fase cruciale. La procedura fonde le catene presenti in `A` e `B` e le scrive nella lista `L`.
  - Riprendendo l'esempio precedente, la procedura fonde la prima catena di `A` (`[3, 5]`) con la prima catena di `B` (`[2, 4, 8]`), producendo `[2, 3, 4, 5, 8]`. Questa è la *prima* nuova catena in `L`.
  - Successivamente, fonde la seconda catena di `A` (`[1, 9]`) con la "seconda" di `B` (che è vuota). Il risultato è `[1, 9]`.
  - Poiché `1` (inizio della seconda catena) è minore di `8` (fine della prima catena), la fusione ha creato **due** catene distinte.
  - Alla fine, `L = [2, 3, 4, 5, 8, | 1, 9]` e `numero_catene` sarà impostato a `2`.

Condizione di Terminazione

- `until numero_catene = 1`: Il ciclo principale si ripete (rieseguendo distribuzione e fusione sulla nuova lista `L`) finché la procedura `merge` non certifica che il risultato della fusione è composto da un'unica, singola catena.
- Quando `numero_catene` è uguale a 1, significa che l'intera lista `L` è stata fusa in un'unica sequenza ordinata, e l'algoritmo termina.

Considerazioni sull'Efficienza

Questo algoritmo è particolarmente efficiente ($O(nlogk)$) quando la lista di input è "quasi ordinata", ovvero contiene un basso numero k di catene naturali. Nel caso peggiore (lista ordinata al contrario), la sua efficienza converge a quella del Merge Sort standard, $O(nlogn)$.

1. La Fase di Distribuzione (`distribuisci`)

Questa procedura ha il compito di svuotare la lista principale `L` e smistare le sue catene, in modo alternato, nelle due liste ausiliarie `A` e `B`.
Logica: La procedura scorre `L` e, finché non è vuota, chiama `copiaCatena` una volta per `A` e una volta per `B`, alternando la destinazione .

```
distribuisci (l: di tipo lista; a e b: di tipo lista per riferimento);
    pl = primolista(l)
    pa = primolista (a)
    pb = primolista (b)
    repeat
```

```
        copiacatena (pl, l, pa, a) //copia una catena da L ad A
        if not finelista(pl, l) then
            copiacatena(pl, l, pb, b) //copia una catena da L a B
    until finelista (pl, l) //ripeti se ci sono elem. in L
```

2. La Fase di Fusione (`merge`)

Questa procedura gestisce la fusione delle catene da `A` e `B` riversandole in `L`. Ha anche il compito cruciale di contare quante nuove catene vengono create.
Logica:

1. Fonde a coppie le catene (`fondiCatena`) finché entrambe le liste `A` e `B` ne hanno ancora.
2. Terminata una delle due liste, copia tutte le catene *rimanenti* dall'altra lista (tramite `copiaCatena`) direttamente in `L` .
3. Incrementa `numero_catene` per ogni catena (sia fusa che copiata) prodotta in `L`.

```
merge(A e B: di tipo lista; L: di tipo lista per riferimento;
    numero_catene : di tipo integer per riferimento )
    pa = primolista(A)
    pb = primolista(B)
    pl = primolista(L)
    while not finelista(pa,A) and not finelista(pb,B) do
        fondiCatena(pa, A, pb, B, pl, L)
        numero_catene = numero_catene + 1
    while not finelista(pa,A) do
        copiaCatena(pa, A, pl, L);
        numero_catene = numero_catene + 1
    while not finelista(pb,B) do
        copiaCatena(pb, B, pl, L)
        numero_catene = numero_catene + 1
```

3. Il Cuore della Fusione (`fondiCatena`)

Questa è la procedura centrale dove avviene la fusione vera e propria di *una* catena da `A` e *una* da `B` .
Logica:
Confronta l'elemento corrente di `A` con quello di `B` e chiama `copia` per spostare il minore dei due in `L` . Se la procedura `copia` rileva che la catena da cui stava copiando è terminata, `fondiCatena` chiama `copiaCatena` per accodare rapidamente l'intera catena rimanente dall'altra lista.

```
fondiCatena(pa: ...; A: ...; pb: ...; B: ...; pl: ...; L: ...)
 finecatena = false;
  repeat
  if leggilista(pa, A) < leggilista (pb, B) then
   copia(pa, A, pl, L, finecatena)
       if finecatena then
       copiaCatena (pb, B, pl, L)
   else
    copia(pb, B, pl, L, finecatena)
    if finecatena then
    copiacatena(pa, A, pl, L)
until finecatena
```

4. Le Procedure Utilitarie (`copiaCatena` e `copia`)

Queste due procedure sono gli strumenti di basso livello che eseguono lo spostamento fisico dei dati e il rilevamento della fine di una catena.
Logica: Chiama `copia` in un ciclo `repeat...until` finché la procedura `copia` non segnala (tramite la variabile `finecatena`) che la catena è terminata.

```
//copia da H a K fino a quando non finisce la catena
 copiaCatena (ph: posizione per riferimento; H: lista;
              pk: posizione per riferimento; K: lista);
    finecatena = false;
    repeat
       copia (ph, H, pk, K, finecatena)
    until finecatena
```

Logica: È la procedura più atomica.

1. Copia un singolo elemento da `X` a `L` .
2. Avanza i puntatori `px` e `pl` .
3. Controlla se la catena sorgente `X` è terminata: o perché la lista è finita (`finelista(px, X)`) o perché l'ordine si è "rotto" (`elemento > leggilista(px, X)`) .
   Aggiorna `finecatena` di conseguenza.

```
//copia un elemento da X ad L e individua la fine della catena
copia (px: posizione per riferimento; X: lista;
       pl: posizione per riferimento; L: lista per riferimento;
       finecatena: boolean per riferimento); elemento = leggilista(px, X)
//copia elemento
   inslista(elemento, pl, L)
```

```
   px = succlista(px, X)
   pl = succlista(pl, L)
   if finelista(px, X) then
     finecatena = true
    else
    //se l'elem. copiatusao è > del successivo la catena è finita
    finecatena = (elemento > leggilista (px, X))
```

Realizzazioni liste in C++

Classe Libro

```
//FILE libro.h

#ifndef _LIBRO_H
#define _LIBRO_H

#include <string>
#include <iostream>

using namespace std;

class Libro{
    public:
        Libro();
        Libro(string);
        void setTitolo(string);
        string getTitolo() const;
        bool operator==(Libro);
    private:
        string titolo;
};
#endif //_LIBRO_H
```

```
//FILE libro.cpp

#include "libro.h"

Libro::Libro(){titolo="";}
Libro::Libro(string t){setTitolo(t);}

void Libro::setTitolo(string t){
```

```
    titolo=t;
}

string Libro::getTitolo() const{
    return (titolo);
}

//sovraccario dell'operatore booleano
bool Libro::operator==(Libro l){
    return (getTitolo()==l.getTitolo());
}
```

Implementazione sequenziale della classe Lista

```
//FILE .h

#ifndef _LISTAV_H
#define _LISTAV_H

//lunghezza massima della lista
const int DIMENSIONE = 1024;

//classe Lista
class Lista{
    public:
        typedef Libro tipoelem;
        typedef int posizione;
        Lista(); //costruttore
        ~Lista(); //distributore

        //operatori
        void creaLista();
        bool listaVuota() const;
        tipoelem leggiLista(posizione) const;
        void scriviLista(tipoelem, posizione);
        posizione primoLista() const;
        bool fineLista(posizione) const;
        posizione succLista(posizione) const;
        posizione predLista(posizione) const;
        void insLista(tipoelem,posizione);
        void canclLista(posizione);
    private:
        tipoelem elementi[DIMENSIONE];
```

```
        int lunghezza;
};
#endif //_LISTAV_H
```

La lista usa un implementazione **sequenziale**, che usa un array **statico** per memorizzare gli elementi.
La struttura:

- Nella riga *10 - 11* si vincola *tipoelem*, ovvero la lista funzionerà solamente se all'interno sarà popolata da oggetti della classe *Libro*.
- `private`:
  - `tipoelem elementi[DIMENSIONE];` : E' la dimensione dell'array fisico vero e proprio, impostato a 1024, tramite costante fissa, dettando così la capacità massima della lista.
  - `int lunghezza` : Questa variabile tiene traccia di **quanti** elementi sono presenti nella lista in un preciso momento.
- `public`:
  - `operatori` : Sono l'insieme delle operazioni pubbliche che si possono eseguire **esternamente** per manipolare la lista.
  - `typedef int posizione;` : Rappresenta semplicemente l'indice dell'array per quanto riguarda la lista.
- **I limiti**:
  L'implementazione di questa lista presenta due **limiti fondamentali** che motivano gli argomenti:
  1. **Tipo vincolato**: La riga `typedef Libro tipoelem;` **costringe** la classe *Lista* a funzionare solamente con **oggetti di tipo *Libro***. Si definisce una Lista **fortemente accoppiata**.
  2. **Dimensione fissa**: La lista conterrà solamente 1024 elementi.

```
//FILE listav.cpp

#include "lista.h"
Lista::Lista({crealista();})
Lista::~Lista(){};

void Lista::creaLista(){lunghezza=0;}

bool Lista::listaVuota() const{
    return(lunghezza==0);
}

Lista::posizione Lista::primoLista() const{
```

```
    return(1); // e quindi pos(1)=pos(n+1) se la lista è vuota
}

Lista::posizone Lista::succLista(posizione p) const{
    if((0<p) && (p<lunghezza+1)) //precondizione
     return (p+1);
    else return(p); // In realtà sarabbe ottimo sollevare un eccezione
}

Lista::posizione Lista::predLista(posizione p) const{
    if((1<p) && (p<lunghezza+1)) //precondizione
     return (p+1);
    else return(p); // vale la stessa regola del successivo
}
```

Come possiamo notare i codice chiama prima il **costruttore** quando si crea un nuovo oggetto `Lista` e a sua volta chiama il metodo `creaLista()`, dove successivamente imposterà la ***lunghezza* a 0**, in modo tale da creare una **lista vuota**.
Prima di far ciò controlla anche se ci sono spazzi di memoria dinamica da liberare tramite l'uso del **distruttore**.
Successivamente controlla se la Lista è effettivamente vuota se `lunghezza==0`. Dopo di che si comincia ad implementare la lista con il primo valore, `primoLista` sarà sempre implementato con 1 dato che si usa la logica degli array, dove la prima posizione è **sempre** 1.

- `Lista::posizone Lista::succLista(posizione p) const`, permette l'implementazione di `succLista`, dove banalmente il nuovo dato da inserire sarà nella posizione successiva a p quindi $p+1$.
- `Lista::posizione Lista::predLista(posizione p) const`, permette l'implementazione di `predLista`, dove banalmente la posizione precedente al punto in cui si trova l'array in posizione p è $p-1$.

```
// FILE listav.cpp 2° parte

bool Lista::fineLista(posizione p) const{
    if((0<p) && (p<=lunghezza+1)) // precondizione
     return (p=lunghezza+1);
    else return(false);
}

tipoelem Lista::leggiLista(posizione p) const{
    if((0<p) && (p<lunghezza+1)) //precondizione
     return(elementi[p-1]);
```

```
}

void Lista::scriviLista(tipoelem a, posizione p){
    if((0<p) && (p<lunghezza+1)) //precondizione
     elementi[p-1]=a;
}

void Lista::insLista(tipoelem a, posizione p){
    if((0<p) && (p<=lunghezza+1)){
        for(int i=lunghezza;i>=p;i--){
            elementi[i]=elementi[i-1];
        }
        elementi[p-1]=a;
        lunghezza++;
    }
}

void Lista::cancLista(posizione p){
    if((0<p) && (p<lunghezza+1))
    if(!listaVuota()){
        for(int i=p-1;i<(lunghezza-1);i++){
            elementi[i]=elementi[i+1];
        }
        lunghezza--;
    }
}
```

Qui ci si addentra su delle implementazioni più specifiche e dettagliate della Lista:

- `bool Lista::fineLista(posizione p) const`, permette l'implementazione di `fineLista`, esso controlla se una posizione p è la medesima (o precedente) posizione dell'ultimo elemento della lista il quale è in posizione `lunghezza` per tanto l'ultimo elemento si troverà in posizione `lunghezza+1`, in base a ciò che sarà presente restituirà la posizione o falso.
- `tipoelem Lista::leggiLista(posizione p) const`, permette l'implementazione di `leggiLista`, questa è la funzione chiave per poter accedere; poiché è vero che la lista ha una lunghezza che varia da 1 a 1024, come visto prima, ma per potervi accedere usiamo gli indici dell'array `elementi` che vanno da 0 a $lunghezza - 1$, per poter leggere un elemento in posizione p bisogna accedere all'indice `elementi[p-1]`.
- `void Lista::scriviLista(tipoelem a, posizione p)`, implementa similmente a `leggiLista`, per scrivere nella posizione logica p, deve modificare l'indice fisico `elementi[p-1]`.

- `void Lista::insLista(tipoelem a, posizione p)` : Questa è l'operazione più costosa e ci permette di inserire un elemento in posizione p :
  1. Esegue un ciclo for che parte dalla fine (lunghezza) e scende fino a p .
  2. Ad ogni passo, **sposta** l'elemento `elementi[i-1]` a **destra**, in `elementi[i]` .
  3. Questo "crea un buco" all'indice $p-1$, dove viene inserito il nuovo elemento a.
  4. Infine, incrementa `lunghezza` .
- `void Lista::cancLista(posizione p)` : Anche questa è un'operazione costosa. Per cancellare un elemento in posizione `p` :
  1. Controlla se la lista non è vuota.
  2. Esegue un ciclo `for` che parte dalla posizione da cancellare (`p-1`) fino alla fine della lista.
  3. Ad ogni passo, **sposta l'elemento** `elementi[i+1]` **a sinistra**, in `elementi[i]` , sovrascrivendo di fatto l'elemento da cancellare.
  4. Infine, decrementa `lunghezza` .

Funzioni di servizio

```
//FILE serviziolv.h

#ifndef _SERVIZIOLV_H
#define _SERVIZIOLV_H

#include <lista.h>

void stampaLista(Lista &);
void epurazioneLista(Lista &);
...

#endif //_SERVIZIOLV_H
```

```
//FILE serviziolv.cpp

#include <iostream>
#include <lista.h>

using namespace std;

void stampaLista(Lista &l){
    cout<<"[";
    Lista::posizione pos;
    for(pos=l.primoLista(); !fineLista(pos);pos=l.succLista(pos)){
        cout<<l.leggiLista(pos);
```

```
        if(!l.fineLista(l.succLista(pos)))
        cout<<",";
    }
    cout<<"]\n";
}

void epurazioneLista(Lista &l){
    Lista::posizione p,q,r;

    p=l.primoLista();
    while(!l.fineLista(p)){
        q=l.succLista(p);
        while(!l.fineLista(q)){
            if(l.leggiLista(p)==l.leggiLista(q)){
                r=l.succLista(q);
                l.cancLista(q);
                q=r;
            }
            else q=l.succLista(q);
        }
        p=l.succLista(p);
    }
}
```

- **`stampaLista(Lista &l)`**:
  È una funzione esterna che riceve una `Lista` (`l`) come parametro.
  La scorre usando i metodi pubblici della lista (`primoLista` , `fineLista` , `succLista`) e stampa ogni elemento che legge (`leggiLista`).
- **`epurazioneLista(Lista &l)`** : È un algoritmo più complesso che rimuove i duplicati.
  Usa un **doppio ciclo** per scorrere la lista: un puntatore `p` scorre ogni elemento dall'alto, e un puntatore `q` scorre gli elementi *successivi* a `p` dal basso.
  Punto chiave: Esegue il confronto `if (l.leggiLista(p) == l.leggiLista(q))` .
  Questo `==` è l'**operatore `operator==` che abbiamo definito nella classe `Libro`** all'inizio.
  Se trova un duplicato (il confronto è `true`), salva quel valore di q in r e usa il metodo pubblico `l.cancLista(q)` per rimuoverlo dalla lista, dopo di che rimette in q l'elemento in r per verificare se è la fine o meno della lista e poter continuare da quel punto.

Funzioni di Test

```
//FILE testlista.cpp

#include <stdio.h>
```

```
#include <iostream>
#include <listav.h>
#include <serviziolv.h>

using namespace std;
int main(){
    Lista l;
    Lista::posizione posElemento=l.primoLista();
    Libro libro;

    libro.setTitolo("Primo");
    l.insLista(libro,posElemento=l.succLista(posElemento));
    libro.setTitolo("Secondo");
    l.insLista(libro,posElemento=l.succLista(posElemento));
    cout<<"\nL\'attuale lista e\':";
    stampaLista(l);
    cout<<"\nOra inserisco l\'elemento Dieci nella seconda posizione...\n";
    libro.setTitolo("Dieci");
    l.insLista(libro,l.succLista(l.primoLista()));
    cout<<"\nLista inserita: "<<endl;
    stampaLista(l);
    cout<<"\nEpurazione lista: "<<endl;
    epurazioneLista(l);
    cout<<"\nEpurazione lista: "<<endl;
    epurazioneLista(l);
    stampaLista(l);
    return 0;
}
```

Overloading degli operatori

L'overloading degli operatori è una caratteristica del C++ che consente di **ridefinire il comportamento di un operatore**(come `+`, `-`, `==`, `<<`) affinché possa essere utilizzato con oggetti di una classe definita dall'utente (come la classe `Libro` vista in precedenza). Per utilizzare un operatore su un oggetto di una classe, è necessario che tale operatore sia stato sovraccaricato per quella classe.

Come definirlo

Si definisce scrivendo una funzione membro (o `friend`) il cui nome è composto dalla parola chiave `operator` seguita dal simbolo dell'operatore che si intende sovraccaricare.

Esempio visto in precedenza con la classe `Libro`: `bool operator==(Libro l);`

Il C++ impone regole precise su cosa si può e non si può fare con il sovraccarico.

1. Operatori NON Sovraccaricabili
Non tutti gli operatori possono essere ridefiniti. Gli operatori esclusi sono fondamentali per il funzionamento del linguaggio stesso:

- `.` (Accesso a membro)
- `.*` (Accesso a membro tramite puntatore)
- `::` (Risoluzione di visibilità)
- `?:` (Operatore condizionale ternario)
- `sizeof` (Operatore di dimensione)

2. Operatori Sovraccaricabili
La maggior parte degli altri operatori, inclusi quelli aritmetici, logici, di confronto e anche operatori più specifici, può essere sovraccaricati :

- Aritmetici: `+`, `-`, `*`, `/`, `%`, `++`, `--`
- Confronto: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Logici: `&&`, `||`, `!`
- Allocazione: `new`, `delete`
- Accesso: `[]` (accesso array), `()` (chiamata di funzione)

3. Regole Fondamentali
Ci sono restrizioni fondamentali che non possono essere violate:

- **Arità:** Non è possibile cambiare il numero di operandi di un operatore (es. un operatore binario come `+` deve rimanere binario; non può diventare unario).
- **Precedenza e Associatività:** Non è possibile cambiare l'ordine di valutazione. L'operatore `*` avrà sempre la precedenza sull'operatore `+`, anche se sovraccaricati.
- **Nuovi Operatori:** Non è possibile inventare nuovi simboli per gli operatori (es. non si può creare un `operator**`).
- **Tipi Primitivi:** Non è possibile sovraccaricare gli operatori per i tipi di dati fondamentali (primitivi) del C++. Ad esempio, non si può ridefinire come funziona l'addizione tra due `int`.

Sintassi ed Esempi

- **Operatori Unari:** Agiscono su un singolo operando (es. `!myString`).

```
class String {
public:
  bool operator!() const;
};
```

- **Operatori Binari:** Agiscono su due operandi (es. `y += z`).

```
class String {
public:
  const String &operator+=(const String & );
};
```

Quando il compilatore incontra l'espressione `y += z`, la traduce internamente in una chiamata al metodo membro: `y.operator+=(z)`.

- **Overload dell'operatore `<<` (Stampa):** Un caso particolare comune è il sovraccarico dell'operatore di stampa `<<` (per `std::cout`). Poiché il primo operando è di tipo `ostream` (e non un oggetto della nostra classe), questo operatore viene tipicamente implementato come una funzione `friend` (amica) della classe.

```
friend ostream& operator<<(ostream& os, const MiaClasse& miaistanza);
```

Ereditarietà e Classi astratte

Per separare l'**interfaccia** della `Lista` (l'elenco di operazioni che *deve* fornire) dalla sua **implementazione** (il *come* queste operazioni vengono realizzate, ad esempio con un vettore o con puntatori), il C++ utilizza due concetti fondamentali: l'**ereditarietà** e le **classi astratte**.

Ereditarietà

L'ereditarietà è un meccanismo che permette a una classe (detta **classe derivata** o sotto-classe) di ereditare metodi e campi da un'altra classe (detta **classe base** o super-classe) . Questo permette alla classe derivata di accedere ai membri `protected` della classe base.

Classe Astratta (o Interfaccia)

Una classe astratta è un tipo speciale di classe base che serve a definire un'**interfaccia**. Contiene metodi (chiamati **funzioni virtuali**) che sono dichiarati ma non hanno un'implementazione.
Regola Fondamentale: Una classe astratta **non può essere istanziata** (non si può creare un oggetto direttamente da essa). Solo le classi effettive possono essere istanziate.
Obiettivo: Per poter creare un oggetto, una classe effettiva deve ereditare dalla classe astratta e **fornire un'implementazione** per *tutte* le funzioni virtuali che la classe base ha lasciato "in sospeso".

La Soluzione: `Lista` come Classe Astratta Template

Combinano questi concetti per creare una definizione di `Lista` potente e flessibile, risolvendo entrambi i problemi dell'implementazione iniziale.

La nuova definizione è:

```
template< class T >
class Lista
{
public:
  typedef T tipoelem;
  typedef int posizione;
  // operatori
  virtual void creaLista() = 0;
  virtual bool listaVuota() const = 0;
  virtual tipoelem leggiLista (posizione) const = 0;
  virtual void scriviLista (tipoelem, posizione) = 0;
  virtual posizione primoLista() const = 0;
  // ... e tutti gli altri metodi
  virtual void cancLista (posizione) = 0;
};
```

Lo $= 0$ sta a significare che il metodo **deve** essere implementato dalle classi derivate, affinché siano astratte anch'esse.
Se lo $= 0$ non viene posto il metodo può essere **reimplementato**. Dato che può esserlo ma non è necessario, è comunque obbligatorio fornire una implementazione nella classe padre. Se non esistono implementazioni generali nella classe padre allora bisognerà indicare $= 0$ accettandoli come metodi virtuali.

Questa classe ha due caratteristiche fondamentali:

- È un **Template**: La dicitura `template< class T >` (che significa "modello") rende la classe **generica**. `T` è un segnaposto per un tipo di dato qualsiasi (come `int`, `double`, o la nostra `Libro`). La riga `typedef T tipoelem;` definisce `tipoelem` come un alias per questo tipo generico `T`.
  Vantaggio: Non siamo più vincolati a soli oggetti di `Libro`. Possiamo ora creare `Lista<Libro>`, `Lista<int>`, `Lista<double>`, ecc.. Non serve più scrivere *tipoelem* ma si potrà usare direttamente `T`.
- È una **Classe Astratta**: Tutti i metodi della `Lista` (gli "operatori" dell'ADT) sono dichiarati come **virtuali puri**. La sintassi `virtual ... = 0;` (ad esempio, `virtual void creaLista() = 0;`) indica che la funzione è virtuale pura.
  Questo significa che la classe `Lista` **non fornisce alcuna implementazione** per questi metodi, ma *obbliga* le classi derivate a implementarli.
  Vantaggio: La classe `Lista` ora definisce solo l'**interfaccia** (il contratto), separandola

dall'implementazione (che sarà definita nelle classi derivate come `vetLista` o `circLista`).

Rendiamo la Lista una classe astratta template

Un template di classe Lista può essere la base per molte classi **Lista** come **Lista Double**, **Lista Int**, **Lista di Libri** ecc...
`template< class T>` trasforma inizialmente la classe in un generico modello, così da risolvere il limite di oggetti di un solo tipo. Il parametro `T` indica il tipo di classe da creare; il tipo di elemento da memorizzare in **Lista** è menzionato genericamente come `T` in tutta l'intestazione della classe **Lista** e nelle definizioni delle funzioni membro.

Sintassi del Template

Definizione della classe template

```
template <typename variabile_tipo>
 class NomeClasse{
     caratteristiche
 }
```

e della funzione membro template

```
templeate <typename variabile_tipo>
tipo_restituito Nome_Classe <variabile_tipo>::
    nome_funzione(parametri) const_opt {
    istruzioni
}
```

Grazie all'uso del template possiamo notare alcune migliorie e caratteristiche, infatti i template sono gestiti **staticamente**, quindi a livello di **compilazione** e non comportano alcun costo in fase esecutiva.
Permette al programmatore di scrivere un codice ancora più **generico** senza preoccuparsi di doverlo cambiare in base alle possibili variazioni di **tipi** a cui il codice va applicato.
Perciò si possono creare **classi identiche strutturalmente** ma che gestiscono solo **tipi degli argomenti e/o tipi dei membri differenti**.
La **Libreria Standard del C++ (STL)** è piena di classi template, dette **classi contenitore** (es. `vector`, `list`, `map`).

- Questo collega la teoria alla pratica. I template non sono un concetto accademico esoterico; sono il fondamento della Standard Template Library (STL) del C++. Quando in C++ si scrive `std::vector<int>` o `std::map<string, double>`, si sta usando una classe template fornita dalla libreria standard, proprio come stiamo facendo noi con `Lista<T>`.

La definizione (il `.h`) e l'implementazione (il `.cpp`) di un template **devono risiedere nello stesso file**, non è possibile avere un header. Questa è una regola tecnica fondamentale e una differenza chiave rispetto alle classi normali. Il compilatore genera il codice per `Lista<int>` quando la incontra. Per farlo, ha bisogno di vedere non solo la *dichiarazione* (i prototipi dei metodi) ma anche l'*implementazione* (il codice sorgente dei metodi). Se l'implementazione fosse in un file `.cpp` separato, il compilatore non potrebbe vederla nel momento in cui compila il file che usa `Lista<int>`, portando a errori in fase di "link".

Vettore a dimensione dinamica

Grazie ai template si è risolto il problema di stabilire un tipo di elemento per il vettore, ora bisogna capire come stabilire e stimare la dimensione massima del vettore per comprendere quanti elementi può ospitare. La soluzione a questo problema è aumentare **dinamicamente** la grandezza del vettore quando è necessario.
Per tanto una possibile soluzione sarebbe definire due vettori a e $a' > a$, dove a è il vettore iniziale e in caso serva più spazio copiare gli elementi di a in a', infine si cambia il valore dell'array iniziale in modo che faccia riferimento al nuovo vettore.

```
template <class T>
void cambiaDimensione(T*& a, int vecchiaDim, int nuovaDim){
    if(nuovaDim<0) throw illegalParameterValue("La nuova lunghezza deve
essere    >=0");

    T* temp=new T[nuovaDim];
    int number;
    if(vecchiaDim<nuovaDim) then
     number = vecchiaDim;
    else number = nuovaDim;
    for(int i=0; i<number;i++){
        temp[i]=a[i];
    }
    delete []a;
    a=temp;
}
```

La Funzione `cambiaDimensione`:
Questa è una funzione template generica (`template<class T>`) che esegue i 3 passi di ridimensionamento:

- `void cambiaDimensione(T*& a, ...)`: Questa è la parte più importante. Il parametro `T*& a` è un **puntatore (`*`) passato per riferimento (`&`)**. Questo significa che se la funzione modifica `a` (facendolo puntare a un nuovo blocco di memoria), la modifica sarà visibile anche all'esterno della funzione.

- `T* temp = new T[nuovaDim];` : Questo è il **Passo 1**. Alloca dinamicamente in memoria un nuovo array (`temp`) della dimensione richiesta.
- `for (...) temp[i]=a[i];` : Questo è il **Passo 2**. Copia i dati dal vecchio array `a` al nuovo array `temp`.
- `delete [] a;` : Libera la memoria occupata dal *vecchio* array `a` per evitare memory leak.
- `a = temp;` : Questo è il **Passo 3**. Fa in modo che il puntatore originale `a` (quello della nostra classe `vetLista` (`LISTAV`)) ora punti al nuovo array `temp`.

Il Concetto di "Array Doubling"

La *strategia* utilizzata ha un nome specifico detto **array doubling** (raddoppio dell'array).

- **Cosa significa:** Quando la nostra lista si riempie, invece di aggiungere solo *un* nuovo spazio (che sarebbe molto inefficiente), chiamiamo `cambiaDimensione` e chiediamo di creare un array di **dimensione doppia** rispetto a quella attuale.
- **Perché:** Questa strategia è molto efficiente. Sebbene una singola operazione di raddoppio possa essere costosa (perché deve copiare tutti gli elementi), "spalma" il costo su molte inserzioni successive, rendendo il costo *medio* ammortizzato di un inserimento molto basso.

La nuova classe LISTAV chiamata ora VETLISTA

```
#ifndef _VETLISTA_H
#define _VETLISTA_H

template< class T >
class vetLista : public Lista<T>
{
  public:
    typedef int posizione;
    vetLista(int dim = 10);          // costruttore
    vetLista(const vetLista<T>&);        // costruttore di copia
    ~Lista();                        //distruttore
    // operatori
    void creaLista();
    bool listaVuota() const;
    tipoelem leggiLista(posizione) const;
    void scriviLista(tipoelem, posizione);
    posizione primoLista() const;
    bool fineLista(posizione) const;
    posizione succLista(posizione) const;
    posizione predLista(posizione) const;
    void insLista(tipoelem, posizione);
    void cancLista(posizione);
 protected:
   tipoelem* elementi;     // vettore
   int lunghezza;          // lunghezza della lista
   int dimensione;  // dimensione del vetore
};
```

Eredita da Lista

```
// il costruttore crea un vettore la cui dimensione è dimIniziale che di default è 10
template< class T >
vetLista< T >::vetLista(int dim) {
   dimensione = dim;
   elementi = new T[dimensione];
   crealista();
}

/* il costruttore di copia effettua una copia o clone di un oggetto. Questo costruttore viene invocato,
 * per esempio, quando un oggetto viene passato per valore ad una funzione o quando una funzione
 * restituisce un oggetto. */
template< class T >
vetLista< T >::vetLista(const vetLista<T>& Lista) {
   dimensione = Lista.dimensione;
   lunghezza = Lista.lunghezza;
   elementi = new T[dimensione];
   for (int i=0; i<Lista.dimensione; i++)
      elementi[i] = Lista.elementi[i];
}

template< class T >
void vetLista< T >::creaLista(){ lunghezza = 0; }

template< class T >
bool vetLista< T >::listaVuota() const { return(lunghezza == 0); }

...
...

#endif // _VETLISTA_H
```

vetLista.h

Rappresentazione collegata circolare con sentinella realizzata mediante doppi puntatori

```
#ifndef _CELLALP_H
#define _CELLALP_H

template< class T >
class Cella{
  public:
    typedef T tipoelem;
    Cella() {}
    Cella(const T& elemento){ this->elemento = elemento; }
    void setElemento(tipoelem e){ elemento = e; }
    tipoelem getElemento() const {return elemento; }
    void setSucc(Cella *p){ succ=p; }
    Cella * getSucc() const{ return succ;}
    void setPrec(Cella *p) { prec=p;}
    Cella * getPrec() const{ return prec;}

  private:
    tipoelem elemento;
    Cella * prec;
    Cella * succ;
};

#endif // _CELLALP_H
```

cellalp.h

Classe Cella

parte pubblica
- setElemento e getElemento
- setSucc e setPrec
- getSucc e getPrec

parte privata
- elemento
- prec
- succ

La nuova classe Lista

FILE listap.h:

```
#ifndef _LISTAP_H
#define _LISTAP_H

#include "cella.h"


template<class T>
class circLista : public Lista<T> {
  public:
    circLista();                          // costruttore
    circLista(const circLista<T>&);       // costruttore di copia
    ~circLista();                         // distruttore
    /* posizione è un puntatore a cella */
    typedef Cella<T> * posizione;         // notare una diversa implementazione del concetto di posizione
    /* Prototipi degli operatori */
    void creaLista();
    bool listaVuota();
    tipoelem leggiLista(posizione);
    void scriviLista(tipoelem, posizione);
    posizione primoLista();
    bool fineLista(posizione);
    posizione succLista(posizione);
    posizione predLista(posizione);
    void insLista(tipoelem, posizione&);
    void cancLista(posizione &p);
  private:
    posizione lista;                      //la lista è rappresentata da un puntatore ad oggetto Cella (la prima cella della lista)
};
```

Implementazione classe Lista

```
//FILE circLista.cpp


#include "listap.h"
```

```
template<class T> circLista<T>::circLista(){creaLista();}
template<class T> const circLista<T>::circLista<T>&Lista){/*realizzazione*/}
template<class T> circLista<T>::~circLista(){
    while(lista->getSucc()!=lista->getPrec()) cancLista(lista->getSucc());
    delete lista;
}

template<class T> void circLista<T>::creaLista(){
    tipoelem ElementoNullo;
    lista=new Cella;
    lista->setElemento(ElementoNullo);
    lista->setSucc(lista);
    lista->setPrec(lista);
    //la sentinella punta a sè stessa
}

template<class T> bool circLista<T>::listaVuota() const{
    return((lista->getSucc()==lista) && (lista->getPrecc()==lista));
}

template<class T> circLista<T>::posizione circLista<T>::primoLista() const{
    return lista->getSucc();
}

template<class T> circLista<T>::posizione
circLista<T>::succLista(Lista::posizione p) const{
    return p->getSucc();
}

template<class T> circLista<T>::posizione
circLista<T>::precLista(Lista::posizione p) const{
    return p->getPrecc();
}

template<class T> bool circLista<T>::fineLista(Lista::posizione p) const{
    return (p==lista);
}

template<class T> circLista<T>::tipoelem circLista<T>::leggiLista(posizione
p) const{
    return p->getElemento();
}
```

```
template<class T> void circLista<T>::scriviLista(tipoelem a, posizione p)
const{
    return p->setElemento(a);
}

template<class T> void circLista<T>::insLista(tipoelem a, Lista::posizione
&p) const{
    Lista::posizione temp;

    temp=new Cella;
    temp->setElemento(a);
    temp->setPrec(p->getPrec());
    temp->setSucc(p);
    (p->getPrec()->setSucc(temp));
    p->setPrec(temp);
    p=temp;
}
template<class T> void circLista<T>::cancLista(Lista::posizione &p){
    Lista::posizione temp;

    temp=p;
    (p->getSucc())->setPrec(p->getPrec());
    (p->getPrec())->setSucc(p->getSucc());
    p=p->getSucc();
    delete(temp);
}
#endif // _LISTAP_H
```